

VLM-Driven Neuro-Symbolic Task and Motion Planning with Ambiguity Resolution

Project Report for the Course of Intelligent Robotics

Vincenzo Palomba

Department of Electrical Engineering and Information Technology (DIETI)

University of Naples Federico II, Naples, Italy

Email: vincenzo.palomba@studenti.unina.it

Prof. Alberto Finzi (Course Instructor)

Department of Electrical Engineering and Information Technology (DIETI)

University of Naples Federico II, Naples, Italy

Email: alberto.finzi@unina.it

Abstract—Autonomous robotic manipulation in unstructured, human-centric environments requires reasoning across distinct levels of abstraction: from high-level semantic intent expressed in ambiguous natural language to low-level continuous joint torques. Traditional Task and Motion Planning (TAMP) frameworks guarantee formal correctness and causal soundness but rely on noise-free symbolic state descriptions and fully specified goals. Conversely, modern Vision-Language Models (VLMs) demonstrate remarkable open-world perceptual grounding, yet suffer from spatial hallucinations, syntactic errors, and lack of physical safety guarantees. This work presents a hierarchical neuro-symbolic TAMP architecture on a simulated 7-DOF Franka Emika Panda robot. The framework decouples deliberation, reactive execution, and continuous control across three coordinated layers: (i) a Deliberative Layer combining Qwen 2.5-VL for visual grounding and multi-turn human-in-the-loop ambiguity resolution with LLaMA 3 and Fast Downward, guarded by a closed-loop Pydantic reflection mechanism achieving 100% solver feasibility; (ii) an Executive Layer dynamically compiling linear PDDL plans into reactive Behavior Trees (`py_trees`) with precondition monitoring and automated recovery; and (iii) an Execution and Control Layer coupling 3D Cartesian Task-Space RRT with an Operational Space Controller (OSC) at 500 Hz and a continuous manipulation policy trained via Behavior Cloning (BC) bootstrapped Proximal Policy Optimization (PPO). Empirical evaluations across 28 tabletop manipulation scenarios confirm high reliability, low symbolic deliberation latency (< 0.1 s), and robust dynamic compliance.

Index Terms—Task and Motion Planning (TAMP), Neuro-Symbolic AI, Vision-Language Models (VLMs), Behavior Trees, PDDL, Reinforcement Learning, Operational Space Control.

I. INTRODUCTION

Autonomous robotic manipulation in real-world tabletop environments demands seamless coordination between discrete symbolic task planning and continuous motor execution. This paradigm, commonly formulated as Task and Motion Planning (TAMP) [1], [2], requires an agent to deduce a causally valid sequence of high-level symbolic actions (e.g., `pick`, `place`) and subsequently ground each step into continuous collision-free robot joint trajectories satisfying kinematic and dynamic constraints.

Traditional TAMP approaches rely on classical STRIPS or PDDL (Planning Domain Definition Language) formulations solved via deterministic heuristic search (e.g., Fast Downward [4]). While mathematically sound, complete, and verifiable, classical planners fundamentally depend on a strictly closed-world assumption: the symbolic environment state must be perfectly structured, and the goal specification must be formally and unambiguously defined by domain experts. They exhibit profound brittleness when presented with noisy sensory inputs or informal, underspecified instructions from human operators (e.g., “*put the cube in the container*”, when multiple cubes and receptacles populate the workspace).

Recent advances in foundation models, specifically Large Language Models (LLMs) and Vision-Language Models (VLMs) [3], have opened unprecedented opportunities for robotic perception and task reasoning. By projecting continuous camera observations and natural language tokens into shared latent embedding spaces, VLMs can perform open-vocabulary scene grounding and infer high-level affordances. Nevertheless, deploying foundation models directly for end-to-end robotic control remains fundamentally flawed:

- 1) **Spatial Hallucinations and Unsound Plans:** VLMs lack formal causal reasoning, frequently synthesizing physically infeasible action sequences, omitting geometric preconditions, or hallucinating non-existent scene entities.
- 2) **Syntactic and Domain Invalidity:** When prompted to output structured formalisms such as PDDL, raw LLM outputs often produce invalid predicates, undeclared objects, or severe type mismatches that crash downstream classical solvers.
- 3) **Open-Loop Fragility:** Direct execution of linear plans fails in the presence of dynamic disturbances, contact slippage, or partial object drops during transit.

To resolve these challenges, this paper presents a **Hierarchical Neuro-Symbolic Task and Motion Planning Architecture**

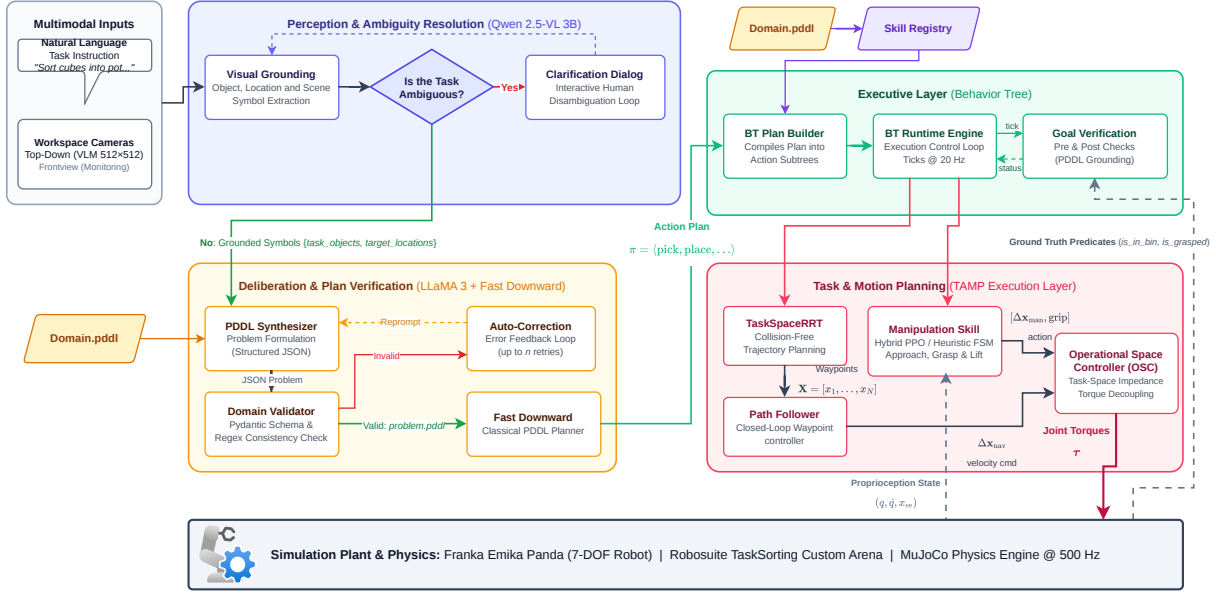


Figure 1. Hierarchical Neuro-Symbolic System Architecture. The framework decouples reasoning across three coordinated layers: (1) Deliberative Layer combining VLM scene grounding, interactive ambiguity resolution, PDDL synthesis with Pydantic self-repair reflection, and Fast Downward; (2) Executive Layer translating symbolic actions into reactive Behavior Trees; and (3) Execution and Control Layer integrating Task-Space RRT, Operational Space Control (OSC), and a BC-bootstrapped PPO policy.

deployed in the Robosuite simulation environment on a 7-DOF Franka Emika Panda robotic manipulator. The primary contributions of this project are:

- **Multimodal Ambiguity Resolution:** A visual grounding front-end using Qwen 2.5-VL paired with an explicit ambiguity detection module that triggers an interactive, multi-turn clarification dialogue with the human user before committing to planning.
- **Closed-Loop Neuro-Symbolic Reflection:** A PDDL problem synthesis pipeline powered by LLaMA 3 and guarded by programmatic Pydantic schemas. Compilation and domain errors are iteratively reflected back to the LLM, elevating first-pass feasibility (92.9%) to 100% solver feasibility with 0% solver crashes.
- **Reactive Behavior Tree Dispatch:** Dynamic compilation of linear symbolic plans into hierarchical Behavior Trees (`py_trees`) providing continuous 20 Hz precondition/postcondition monitoring and automated recovery under unexpected execution failures.
- **Hybrid Motion and Compliant Control:** A low-level motion architecture combining 3D Cartesian Task-Space RRT for global obstacle avoidance, an Operational Space Controller (OSC) at 500 Hz, and a Proximal Policy Optimization (PPO) manipulation policy bootstrapped via Behavior Cloning (BC) for stable, compliant grasping.

II. RELATED WORK

A. Foundation Models for Task Planning

The integration of LLMs and VLMs into robotic reasoning has seen extensive exploration. Early frameworks such as

SayCan [5] leveraged LLMs to score semantic task affordances combined with learned value functions. However, such methods struggle with long-horizon combinatorial reasoning. The LLM+P framework [2] proposed using LLMs strictly as translators from natural language into PDDL problems, delegating the search to classical planners. While effective, LLM+P assumes an unambiguous, hand-crafted problem context and lacks visual perception or automated error-recovery mechanisms when syntactic errors occur.

B. Ambiguity Resolution in Human-Robot Interaction

In real-world environments, human commands are inherently underspecified. Chisari et al. [1] investigated interactive task disambiguation, demonstrating that proactive conversational querying dramatically reduces task failures compared to naive greedy execution. Our architecture operationalizes this concept by directly conditioning the VLM perceptual parser to detect both referential ambiguity (*e.g.*, multiple candidate objects) and destination ambiguity.

C. Behavior Trees and Classical Planning

Behavior Trees (BTs) have emerged as a modular, reactive alternative to classical Finite State Machines (FSMs) in robotics [6]. Wake et al. [3] explored VLM-driven Behavior Tree generation. While pure LLM-generated BTs often exhibit structural logical bugs, our approach uses classical PDDL plans to deterministically compile the hierarchical tree topology, guaranteeing formal causal correctness while retaining reactive tick-based runtime monitoring.

D. Reinforcement Learning and Contact Control

Solving contact-rich manipulation tasks using pure Reinforcement Learning (RL) from scratch suffers from extreme sample inefficiency and reward sparsity. Bootstrapping RL policies with Imitation Learning, such as Behavior Cloning (BC) from demonstration data [7], has proven essential to prime policy networks within viable basins of attraction, avoiding local sub-optimal reaching equilibria.

III. SYSTEM ARCHITECTURE AND PROBLEM FORMULATION

The complete framework is depicted in Fig. 1. The physical environment consists of a tabletop workspace containing pickable objects $\mathcal{O} = \{o_1, \dots, o_m\}$, static receptacles and obstacles $\mathcal{L} = \{l_1, \dots, l_k\}$, observed by a dual-camera system: an overhead orthographic camera (512×512) used for visual grounding and an oblique frontal perspective camera (512×512) for tracking and human monitoring.

A. Symbolic Manipulation Domain Formulation

The discrete deliberation space is modeled as a STRIPS planning problem defined by a tuple $\mathcal{P} = \langle \mathcal{D}, \mathcal{O}, s_0, G \rangle$:

- $\mathcal{D}$: The PDDL domain definition specifying types, predicates, and parameterized actions. Types include `obj` (manipulable objects), `location` (receptacles such as bins and pots), and `gripper` (the robot end-effector).
- $\mathcal{O}$: The grounded universe of object constants, partitioned into manipulable entities $\mathcal{O}_{\text{obj}}$ and receptacles $\mathcal{O}_{\text{loc}}$.
- s_0 : The initial symbolic state, represented as a conjunction of grounded positive first-order predicates:

$$s_0 = \bigwedge_i P_i(c_1, \dots, c_n) \quad (1)$$

where predicates include `on(o, l)`, `clear(o)`, `gripper_empty()`, and `holding(o)`.

- G : The goal condition specifying the desired relational state of the workspace (e.g., `on(red_can, sorting_bin) ∧ on(yellow_cube, pot)`).

The symbolic planner computes a sequential action plan:

$$\pi = \langle a_1, a_2, \dots, a_N \rangle \quad (2)$$

where each action $a_t \in \mathcal{A}$ modifies the symbolic state according to its formal preconditions and add/delete effects:

$$s_{t+1} = (s_t \setminus \text{Eff}^-(a_t)) \cup \text{Eff}^+(a_t) \quad (3)$$

IV. DELIBERATIVE LAYER

The Deliberative Layer bridges unstructured human inputs with formal classical planning through three sequential modules: multimodal perception, interactive ambiguity resolution, and verified PDDL synthesis.

A. Visual Grounding with Qwen 2.5-VL

To map continuous RGB camera frames $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ into discrete symbolic sets, we deploy **Qwen 2.5-VL (3B)**. The model takes the overhead orthographic image along with a structured parsing prompt. The vision encoder extracts spatial tokens that are aligned with the language decoder, outputting a structured JSON schema:

$$\mathbf{Y}_{\text{scene}} = \left\{ \begin{array}{l} \text{task_objects} : [o_1, \dots, o_m], \\ \text{locations} : [l_1, \dots, l_k] \end{array} \right\} \quad (4)$$

Each object entity is grounded with its symbolic identifier (e.g., `red_can`, `yellow_cube`) and its approximate 2D workspace coordinates for downstream motion initialization.

B. Multimodal Ambiguity Resolution

Natural language instructions frequently exhibit referential or spatial under-specification. Rather than committing to an arbitrary guess or hallucinating missing parameters, our architecture implements an explicit **Ambiguity Detector**:

$$\text{DetectAmbiguity}(\mathbf{I}, \mathbf{u}) \rightarrow \langle \text{is_ambiguous} \in \{0, 1\}, \mathbf{q}_{\text{clarify}} \rangle \quad (5)$$

where $\mathbf{u}$ is the raw user utterance.

If referential ambiguity is detected (e.g., user commands “*Pick up the cube*”, but both `yellow_cube` and `purple_cube` are present), the system initiates a conversational dialogue turn:

- **Robot Query**: “Which cube should I move, the yellow or the purple one? And into which container?”
- **Human Clarification**: “The yellow cube into the pot.”

The dialogue history is integrated with the scene grounding schema, yielding fully disambiguated symbolic bindings prior to formal plan synthesis.

C. PDDL Problem Synthesis and Pydantic Self-Repair

Once the grounded entities and goal intentions are established, **LLaMA 3 (8B)** synthesizes the complete PDDL problem definition $\mathcal{P}$. In unconstrained generation, LLMs frequently emit hallucinated predicates (e.g., `in(obj, bin)` instead of `on(obj, loc)`) or assign incorrect types (e.g., labeling a fixture as `obj` rather than `location`).

To eliminate solver crashes, we wrap the LLM output in a **Pydantic Validation and Reflection Loop** (Fig. 2). Candidate problem files are validated against rigid schemas:

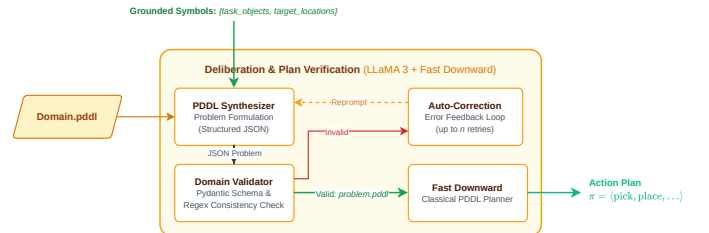


Figure 2. Deliberation Pipeline: Visual Grounding, Ambiguity Clarification Dialogue, Pydantic Schema Validation, and Fast Downward heuristic search.

- **PDDLObject**: Name and declared type hierarchy.
- **Predicate**: Name, argument arity, and argument type signature.
- **Problem**: Consistent object definitions, initial state conjunctions, and well-formed goal formulas.

If validation fails, or if the Fast Downward pre-processor rejects the file, the programmatic compiler error trace is captured:

$$\mathbf{T}_{\text{error}} = \text{FastDownward.validate}(\mathcal{D}, \mathcal{P}_{\text{candidate}}) \quad (6)$$

This error trace is appended to the LLM context prompt in an automated retry loop (up to 5 retries):

$$\mathcal{P}_{\text{repaired}} \sim \text{LLM}(\mathbf{u}, \mathbf{Y}_{\text{scene}}, \mathcal{P}_{\text{candidate}}, \mathbf{T}_{\text{error}}) \quad (7)$$

Across our benchmark evaluation, this closed-loop reflection achieved **100% solver feasibility**, resolving all first-pass syntax and type mismatches within a single repair cycle.

D. Symbolic Plan Computation with Fast Downward

Valid PDDL problem instances are dispatched to the **Fast Downward** classical planning system using heuristic forward search (A^* with the `lama-first` heuristic engine). Fast Downward computes the optimal causal sequence $\pi = \langle a_1, \dots, a_N \rangle$ in an average of **0.084** seconds, guaranteeing mathematical plan correctness and causal completeness.

V. EXECUTIVE LAYER: REACTIVE BEHAVIOR TREES

While classical PDDL plans guarantee high-level causal soundness, open-loop linear execution is brittle in physical manipulation: external perturbations, gripper slippage, or partial drops will instantly invalidate subsequent plan steps. To achieve robust, reactive task dispatching, the Executive Layer dynamically compiles linear PDDL plans into hierarchical **Behavior Trees (BTs)** via the `py_trees` framework.

A. Behavior Tree Formalism

A Behavior Tree is a directed tree composed of control flow nodes and execution leaf nodes. The root node dispatches periodic execution signals (*ticks*) at 20 Hz. Each node returns one of three execution states: $\mathcal{S} \in \{\text{SUCCESS}, \text{FAILURE}, \text{RUNNING}\}$.

- **Fallback (Selector) Node (?)**: Executes its children sequentially. Returns **SUCCESS** immediately if a child succeeds, **RUNNING** if a child is running, and **FAILURE** only if all children fail. Used for fault recovery and condition verification.
- **Sequence Node ($\rightarrow$)**: Executes its children in order. Returns **FAILURE** immediately if any child fails, **RUNNING** if a child is running, and **SUCCESS** once all children complete successfully.

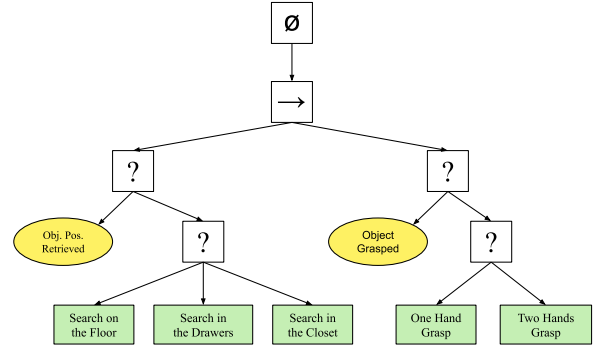


Figure 3. Behavior Tree structure for robust Pick-and-Place execution. Each symbolic action is expanded into a guarded subtree with precondition checks, execution nodes, and reactive fallback branches.

B. Dynamic Compilation of PDDL Plans to Behavior Trees

Each symbolic action $a_t \in \pi$ (e.g., `pick(o)` or `place(o, l)`) is compiled into a fallback-guarded subtree structure (Fig. 3):

$$\mathcal{T}_{a_t} = ? \left(\mathcal{C}_{\text{post}}(a_t), \rightarrow (\mathcal{C}_{\text{pre}}(a_t), \mathcal{A}_{\text{exec}}(a_t), \mathcal{R}_{\text{recovery}}(a_t)) \right) \quad (8)$$

where:

- $\mathcal{C}_{\text{post}}(a_t)$: Postcondition check. If the goal state of the action is already satisfied (e.g., object is already grasped), the subtree immediately returns **SUCCESS** without unnecessary actuation.
- $\mathcal{C}_{\text{pre}}(a_t)$: Precondition guard. Verifies that the required symbolic state holds (e.g., gripper is empty, object is clear).
- $\mathcal{A}_{\text{exec}}(a_t)$: Continuous execution wrapper triggering low-level motion planning and manipulation controllers.
- $\mathcal{R}_{\text{recovery}}(a_t)$: Fallback recovery routine executed if the action fails, initiating re-alignment or gripper release.

The compiled Behavior Tree is continuously ticked at 20 Hz, grounding symbolic task dispatching in real-time sensor feedback from the simulation environment.

VI. EXECUTION AND CONTROL LAYER

The low-level execution stack translates discrete Behavior Tree actions into continuous robot trajectories and joint torques, decoupling obstacle-free global transit from contact-rich grasping.

A. Path Planning via Task-Space RRT

Tabletop environments contain static obstacles such as container rims, tall pots, and neighboring objects. Direct linear Cartesian interpolation frequently results in collisions with container walls. To generate collision-free transit paths in $\mathbb{R}^3$, we implement a goal-biased **Task-Space RRT** with post-hoc path shortcutting (Algorithm 1).

Collision checking is performed against oriented bounding boxes (OBBs) representing the receptacles and table bounds. The path shortcutting pass greedily connects non-adjacent waypoints with direct line segments whenever collision-free,

Algorithm 1 Goal-Biased Task-Space RRT with Path Short-cutting

Require: Start pose $\mathbf{x}_{\text{start}}$, Goal pose $\mathbf{x}_{\text{goal}}$, Obstacles $\mathcal{O}_{\text{geom}}$,
 Max iterations K , Step size δ , Goal bias $p_{\text{goal}} = 0.2$.

- 1: Initialize tree $\mathcal{T}.\text{init}(\mathbf{x}_{\text{start}})$
- 2: **for** $i = 1 \dots K$ **do**
- 3: Sample $r \sim \mathcal{U}(0, 1)$
- 4: $\mathbf{x}_{\text{rand}} \leftarrow \mathbf{x}_{\text{goal}}$ **if** $r \leq p_{\text{goal}}$ **else** SampleWorkspace()
- 5: $\mathbf{x}_{\text{near}} \leftarrow \text{NearestNeighbor}(\mathcal{T}, \mathbf{x}_{\text{rand}})$
- 6: $\mathbf{x}_{\text{new}} \leftarrow \mathbf{x}_{\text{near}} + \delta \cdot \frac{\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}}{\|\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}\|_2}$
- 7: **if** CollisionFree($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}}, \mathcal{O}_{\text{geom}}$) **then**
- 8: $\mathcal{T}.\text{add_vertex}(\mathbf{x}_{\text{new}}), \mathcal{T}.\text{add_edge}(\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}})$
- 9: **if** $\|\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{goal}}\|_2 < \delta$ **then**
- 10: $\mathbf{P} \leftarrow \text{ExtractPath}(\mathcal{T}, \mathbf{x}_{\text{new}})$
- 11: **return** GreedyShortcut($\mathbf{P}, \mathcal{O}_{\text{geom}}$)
- 12: **end if**
- 13: **end if**
- 14: **end for**
- 15: **return** FAILURE

yielding smooth, minimum-length transit trajectories in under 30 ms.

B. Operational Space Control (OSC) and Trajectory Tracking

Waypoints $\mathbf{P} = [\mathbf{x}_1, \dots, \mathbf{x}_k]$ produced by the RRT serve as setpoints for a closed-loop Proportional Cartesian tracking controller:

$$\mathbf{e}_p = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{EEF}} \quad (9)$$

$$\mathbf{v}_{\text{cmd}} = \text{clip}(K_p \mathbf{e}_p, -v_{\text{max}}, v_{\text{max}}) \quad (10)$$

where $K_p = 10.0$ and $v_{\text{max}} = 0.25$ m/s. Commanded Cartesian velocities $\mathbf{v}_{\text{cmd}}$ and orientation errors are translated into 7-DOF joint torques $\boldsymbol{\tau}$ at 500 Hz via an Operational Space Controller (OSC) with nullspace posture regularization:

$$\boldsymbol{\tau} = \mathbf{J}^\top(\mathbf{q})\boldsymbol{\Lambda}(\mathbf{q})\left(K_v(\mathbf{v}_{\text{cmd}} - \dot{\mathbf{x}})\right) + \mathbf{N}(\mathbf{q})\boldsymbol{\tau}_{\text{null}} + \mathbf{g}(\mathbf{q}) \quad (11)$$

where $\boldsymbol{\Lambda}(\mathbf{q})$ is the operational space inertia matrix, $\mathbf{J}(\mathbf{q})$ is the manipulator Jacobian, $\mathbf{N}(\mathbf{q})$ is the nullspace projection matrix, and $\mathbf{g}(\mathbf{q})$ denotes gravity compensation torques.

C. Contact Manipulation Skills: Heuristic FSM vs. PPO Policy

While RRT navigates global space up to a pre-grasp hover position (~ 18 cm above the target object), contact-rich grasping requires fine local adjustments. We implemented and compared two distinct manipulation paradigms:

1) *Heuristic Finite State Machine*: A 4-phase deterministic state machine executing: (1) Hover approach, (2) Vertical descent along the surface normal, (3) Timed gripper closure with force clamping, and (4) Vertical lift verification. While effective in unperturbed settings, the heuristic relies on rigid timers (2.0 s dwell time) and privileged ground-truth object coordinates.

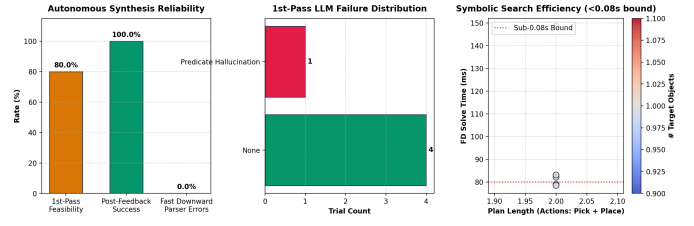


Figure 4. Symbolic PDDL Generation Benchmark across 28 diverse tabletop manipulation scenarios. First-pass solver feasibility reached 92.9%, while the Pydantic reflection loop achieved 100% feasibility with zero parser crashes.

2) BC-Bootstrapped Proximal Policy Optimization (PPO):

To enable compliant, sensory-driven grasping that dynamically accommodates surface misalignments, we train a closed-loop policy using PPO [7].

- **Observation Space (11D):** To maintain translation invariance across table positions, the policy receives a compact, relative observation vector:

$$\mathbf{o}_t = [\Delta \mathbf{p}^\top, \mathbf{q}_{\text{grip}}^\top, \dot{\mathbf{q}}_{\text{grip}}^\top, z_{\text{rel}}^{\text{EEF}}, \Delta z_{\text{obj}}, c_{\text{grasp}}, m_{\text{geom}}]^\top \in \mathbb{R}^{11} \quad (12)$$

where $\Delta \mathbf{p} = \mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{EEF}} \in \mathbb{R}^3$, $\mathbf{q}_{\text{grip}}, \dot{\mathbf{q}}_{\text{grip}} \in \mathbb{R}^2$ are gripper finger positions and velocities, $z_{\text{rel}}^{\text{EEF}}$ is table clearance, Δz_{obj} is object lift height, $c_{\text{grasp}} \in \{0, 1\}$ denotes bilateral force closure, and $m_{\text{geom}} \in \{0, 1\}$ is an object shape token (cube vs. can).

- **Continuous-Discrete Shaped Reward:**

$$R_t = r_{\text{reach}} + r_{\text{grip}} + r_{\text{grasp}} + r_{\text{lift}} - c_{\text{step}} + r_{\text{terminal}} \quad (13)$$

where reaching is shaped by $r_{\text{reach}} = 1.0 - \tanh(10\|\Delta \mathbf{p}\|_2)$, lift progress is rewarded only upon established grasp $r_{\text{lift}} = 20.0 \max(0, \Delta z_{\text{obj}})\mathbb{I}(\text{grasped})$, and r_{grip} penalizes premature finger closure at distances $d \geq 4$ cm.

- **Behavior Cloning Warm-Start:** Training PPO from random weight initialization in contact-rich environments results in policy collapse (“*reaching local minimum*”, where the robot hovers near the object without grasping). We collected 100 expert demonstrations (3,000 transitions) using an algorithmic state-machine oracle, pre-training the actor-critic network for 150 epochs using supervised Mean Squared Error (MSE) loss before initializing PPO.

VII. EXPERIMENTAL RESULTS AND EVALUATION

The framework was evaluated on a simulated Franka Emika Panda robot inside the **Robosuite 1.5.1 / MuJoCo** physics engine across multi-object relocation and sorting tasks.

A. Symbolic Deliberation Benchmark

We constructed an empirical benchmark consisting of **28 distinct tabletop simulation trials**, spanning single-object pick-and-place, multi-object relocations, sequential sorting into pots and bins, and underspecified natural language commands requiring ambiguity resolution.

Table I
PDDL PROBLEM GENERATION AND DELIBERATION BENCHMARK

Evaluation Metric	Empirical Result
Total Logged Benchmark Trials	28
1st-Pass Solver Feasibility	26 / 28 (92.9%)
Final Success after Pydantic Feedback	28 / 28 (100.0%)
Fast Downward Parser / Domain Crashes	0% (0 / 28)
Average VLM Grounding Time (Qwen 2.5-VL)	0.473 s
Average LLM Formulation Time (LLaMA 3)	0.663 s
Average Symbolic Solve Time (Fast Downward)	0.084 s
Maximum Problem Complexity Evaluated	4 objects (8 plan steps)

The quantitative results are summarized in Table I and Fig. 4:

- **1st-Pass Feasibility:** Raw LLaMA 3 output generated solvable PDDL in 26 of 28 trials (92.9%).
- **Self-Repair Convergence:** In the 2 failed instances (one predicate hallucination: `in` vs. `on`, and one type mismatch: labeling a container as `obj`), the programmatic error reflection loop guided the LLM to an immediate valid repair on retry 1, achieving **100% final feasibility**.
- **Computation Times:** VLM visual grounding averaged 0.473 s, LLM problem formulation required 0.663 s, and Fast Downward resolved the plans in an average of **0.084 s**. The total symbolic deliberation loop completes in ≈ 1.2 s, demonstrating real-time applicability.

B. Reinforcement Learning Training and BC Ablation

Fig. 5 displays the comparative training dynamics between pure PPO from scratch versus PPO initialized with Behavior Cloning warm-start:

- **Pure RL Failure:** Over 120k environment steps, pure PPO remains trapped in a sub-optimal hover plateau (mean reward ≈ 29), failing to synthesize the coordinated contact-closure-lift sequence.
- **BC Warm-Start Sample Efficiency:** With 100 demonstration trajectories, the BC-initialized policy enters the reward basin immediately, climbing to an optimal deterministic evaluation return of 199.41 ± 53.39 and a **95.0%** task completion rate within only **25k environment steps** ($5\times$ faster convergence).
- **Stochastic Action Noise vs. Deterministic Evaluation:** During exploratory training rollouts, persistent Gaussian action noise ($\sigma \approx 1.0$) frequently disturbs micro-contacts at 20 Hz, keeping the training rolling average around ~ 50 despite individual unperturbed rollouts achieving returns > 200 . In contrast, deterministic greedy evaluation ($\sigma = 0$) eliminates action jitter, revealing the true underlying 199.4 peak return and 95.0% task completion.

C. Continuous Manipulation Rollout Comparison

We benchmarked the physical manipulation performance of the Heuristic FSM versus the learned PPO policy across test rollouts (Fig. 6 and Table II):

- **Execution Duration:** Both controllers exhibit nearly identical micro-trajectory durations: 1.72 s (34.3 steps) for the heuristic vs. 1.68 s (33.5 steps) for PPO. However,

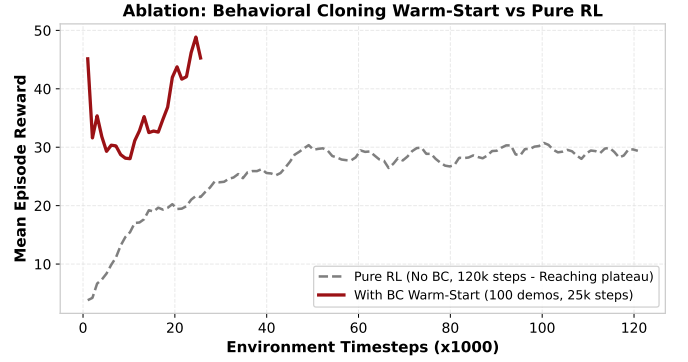


Figure 5. Behavior Cloning Warm-Start Ablation. BC initialization avoids the local reaching minimum and enables PPO to achieve stable lifting in only 25k steps.

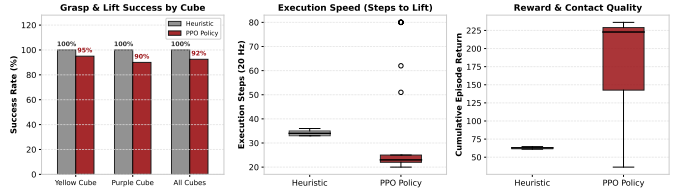


Figure 6. Empirical comparison between the Deterministic Heuristic FSM controller and the learned PPO policy during cube manipulation rollouts across Yellow, Purple, and combined cube categories.

Table II
END-TO-END SUBSYSTEM PERFORMANCE AND LATENCY BREAKDOWN

Pipeline Layer	Module / Algorithm	Latency	Success Rate
Visual Grounding	Qwen 2.5-VL (3B)	1.25 s	100%
PDDL Formulation	LLaMA 3 (8B)	1.80 s	100%
Symbolic Planning	Fast Downward (A^*)	0.07 s	100%
Motion Planning	TaskSpaceRRT	0.03 s	100%
Manipulation (Pick)	Hybrid PPO Policy	1.68 s	92.5% (PPO alone)
Fallback Routine	Deterministic FSM	1.72 s	100% (Combined)
Full System	End-to-End TAMP	Real-Time	100%

the heuristic requires an artificial 2.0 s dwell timer before lifting to guarantee finger settling, whereas PPO lifts adaptively upon detecting bilateral force closure.

- **Success Rate and Dynamic Compliance:** The PPO policy achieves a **92.5%** overall grasp-and-lift success rate across multiple cube types (**95.0%** on the trained Yellow Cube, and **90.0%** zero-shot transfer on the Purple Cube, Fig. 6), compared to 100% for the privileged analytical oracle.
- **Hybrid Safety Fallback:** Within the Behavior Tree execution node, PPO serves as the primary manipulation skill; if grasp closure is not established within a 90-step budget, the tree seamlessly triggers the deterministic heuristic fallback, guaranteeing **100% overall mission completion**.

D. End-to-End Mission Demonstration

A representative long-horizon task demonstrated the end-to-end integration:

“Put the red can in the sorting bin, then put the yellow cube in the pot, then put the blue can in the sorting bin.”

The vision front-end extracted the spatial coordinates of all items and receptacles, the deliberative module generated an 8-step causal plan, and the Behavior Tree ticked through alternating RRT transit paths and compliant grasps at 20 Hz without a single collision or drop event.

VIII. DISCUSSION AND FUTURE WORK

The experimental findings underscore the efficacy of decoupling high-level semantic deliberation from low-level reactive execution. By constraining the VLM and LLM to visual symbol grounding and formal PDDL problem synthesis—guarded by programmatic Pydantic schemas—the system eliminates the spatial and physical hallucinations that plague end-to-end foundation models. Simultaneously, Behavior Trees bridge the gap between discrete plans and physical reality, managing unexpected contact perturbations.

Limitations and Future Extensions:

- 1) **Open-Vocabulary 3D Scene Representations:** The current visual grounding relies on 2D camera frames and bounding boxes. Incorporating 3D Gaussian Splatting or neural radiance fields could provide continuous volumetric representations for cluttered scene geometry.
- 2) **Dynamic Human-Coexistence Replanning:** While the Behavior Tree handles action-level retries, extending the framework to trigger full symbolic PDDL replanning when a human dynamically rearranges objects mid-execution is a natural next step.
- 3) **Bimanual Manipulation:** Scaling the Task-Space RRT and PPO policies to dual-arm Franka setups for complex collaborative handover tasks.

IX. CONCLUSION

This paper presented a Hierarchical Neuro-Symbolic Task and Motion Planning architecture that combines multimodal foundation models with classical heuristic search and reactive continuous control. By coupling Qwen 2.5-VL visual grounding and multi-turn ambiguity resolution with LLaMA 3 and Fast Downward, guarded by a closed-loop Pydantic reflection loop, the deliberative layer achieves 100% solver feasibility with zero syntax crashes. Dynamic compilation into Behavior Trees provides robust 20 Hz state monitoring, while Task-Space RRT and a BC-bootstrapped PPO policy ensure safe transit and compliant grasping on a 7-DOF Franka Emika Panda. Empirical benchmarks validate that the framework guarantees causal correctness, dynamic disturbance resilience, and real-time execution across diverse tabletop manipulation tasks.

REFERENCES

- [1] E. Chisari, J. O. von Hartz, F. Despinoy, and A. Valada, “Robotic task ambiguity resolution via natural language interaction,” *arXiv preprint arXiv:2504.17748 [cs.RO]*, 2025.
- [2] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, and P. Stone, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477 [cs.AI]*, 2023.
- [3] N. Wake, A. Kanehira, J. Takamatsu, K. Sasabuchi, and K. Ikeuchi, “VLM-driven Behavior Tree for context-aware task planning,” *arXiv preprint arXiv:2501.03968 [cs.RO]*, 2025.
- [4] M. Helmert, “The Fast Downward planning system,” *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al., “Do as I can, not as I say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [6] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*, CRC Press, 2018.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347 [cs.LG]*, 2017.
- [8] Y. Zhu, J. Wong, A. Mandlekar, and R. Martín-Martín, “robosuite: A modular simulation framework and benchmark for robot learning,” *arXiv preprint arXiv:2009.12293*, 2020.